

Docker Interview - Questions & Answers

Fresher / 6 Months Experience | Simple conversational answers

SECTION 1 — Basic / Conceptual Questions

Q1.

What is Docker and why do we use it? How is it different from a Virtual Machine?

Answer:

Docker is a tool that packages your application along with everything it needs to run — like the code, runtime, and libraries — into a container. Think of it like a box that holds your app and all its requirements. The main difference from a Virtual Machine is that Docker containers share the host OS kernel, so they are much lighter and faster to start. A VM needs its own full operating system, which takes more memory and time. With Docker, you can run five containers on a machine using very little memory, whereas five VMs would be very heavy.

Q2.

What is a Docker Image and what is a Docker Container?

Answer:

A Docker image is like a blueprint or a recipe. It contains the operating system, runtime environment, all dependencies, and your application code — everything needed to run your app. A Docker container is what you get when you actually run that image. It is the live, running instance. Think of it this way: the image is the recipe, and the container is the cooked meal. One image can create many containers.

Q3.

What is a Dockerfile and what are the key instructions in it?

Answer:

A Dockerfile is a text file with step-by-step instructions that tells Docker how to build your image. Key instructions are: FROM (sets the base image, like Ubuntu or Python), WORKDIR (sets the working directory inside the container), COPY (copies files from your machine into the image), RUN (runs commands during build, like installing packages), ENV (sets environment variables), EXPOSE (declares which port the app uses), CMD (sets the default command to run when the container starts), and ENTRYPOINT (sets the fixed main command for the container).

Q4.

What is the difference between CMD and ENTRYPOINT?

Answer:

CMD sets a default command that runs when the container starts, but it can be easily overridden from the command line. ENTRYPOINT is the fixed main command that always runs and is harder to override. When you use both together, ENTRYPOINT is the main command and CMD provides default arguments to it. For example, if your ENTRYPOINT is python and CMD is app.py, the container always runs Python, but you can change which file it runs by passing a different argument.

Q5.

What is the difference between COPY and ADD in a Dockerfile?

Answer:

COPY is used to copy files from your local machine into the Docker image. It is simple and does exactly what it says. ADD can do the same thing, but it has extra features — it can download files from the internet using a URL, and it can automatically extract tar archive files. Best practice is to always use COPY for

local files because it is clearer and more predictable. Only use ADD when you specifically need to fetch from a URL or extract a tar file.

Q6.

What are Docker layers and why should we minimize them?

Answer:

Every instruction in a Dockerfile creates a new layer in the image. These layers are stacked on top of each other to form the final image. Each layer adds to the total size of the image. This is why we should minimize layers by combining multiple RUN commands into a single RUN instruction using double ampersands. This keeps the image smaller, faster to download, and quicker to deploy. Also, Docker caches layers, so if nothing changed in a layer, it reuses it on the next build, making builds faster.

Q7.

What are Docker Volumes and what is the difference between volumes and bind mounts?

Answer:

Docker containers are temporary — when a container stops, all data inside it is lost. Docker volumes solve this by storing data outside the container so it persists even after the container is destroyed. A volume is managed by Docker and stored in Docker's own storage area. A bind mount is when you manually map a folder from your host machine directly into the container. Use volumes for production and databases because they are more reliable. Use bind mounts during development when you want live code changes to reflect inside the container immediately.

Q8.

What is Docker Networking? Name the different network types.

Answer:

Docker networking controls how containers communicate with each other and with the outside world. The main network types are: Bridge — the default network where containers get their own private IP and can talk to each other; Host — the container shares the host machine's network directly with no isolation; None — the container has no network access at all, completely isolated; Overlay — used with Docker Swarm for containers across multiple machines. Most of the time you will use bridge networking for containers on the same machine.

SECTION 2 — Intermediate Questions

Q9.

What is the difference between docker run, docker build, and docker push?

Answer:

Docker build reads your Dockerfile and creates a Docker image from it. Docker run takes an image and starts a container from it — if the image is not available locally, it first pulls it from a registry. Docker push uploads your locally built image to a registry like Docker Hub so others can use it. The typical workflow is: write Dockerfile, run docker build to create the image, run docker push to upload it to a registry, and then anyone can docker run to start a container from it.

Q10.

What is a Docker Registry? What is the difference between Docker Hub and a private registry?

Answer:

A Docker registry is a centralized storage system where Docker images are stored and shared. Docker Hub is the official public registry run by Docker — anyone can push and pull images there for free. A private registry is one that only your organization or team can access. Examples include Amazon ECR, JFrog Artifactory, and self-hosted registries. If your application code is sensitive or proprietary, you would use a private registry instead of Docker Hub to keep your images secure.

Q11.

What are some best practices for securing Docker containers?

Answer:

Never run containers as the root user — create a non-root user in your Dockerfile. Use minimal base images like Alpine to reduce the attack surface. Regularly scan your images for vulnerabilities using tools like Trivy or Snyk. Never hardcode secrets or passwords in your Dockerfile — use secrets management tools. Set containers to read-only filesystem where possible. Do not use privileged mode unless absolutely necessary. Keep your Docker and base images updated with the latest security patches.

Q12.

What is the difference between a stateless and stateful application in Docker?

Answer:

A stateless application does not remember previous requests. Each request is handled independently. These are easy to containerize because you can spin up or destroy containers freely without worrying about data loss. A stateful application remembers data between requests — like a database. These are trickier in Docker because containers are temporary by nature. To make a container stateful, you use Docker volumes which store data outside the container, so the data persists even when the container is stopped or restarted.

Q13.

What is container orchestration and why is it important?

Answer:

Container orchestration is the automated management of multiple containers across multiple machines. When you have hundreds of containers running in production, you cannot manage them manually. Orchestration tools automatically handle deploying containers to the right servers, scaling them up or down based on traffic, restarting failed containers, and managing networking between them. The most popular orchestration tool is Kubernetes. Docker Swarm is a simpler alternative built into Docker. Amazon ECS is another option on AWS.

Q14.

What is a Multi-Stage Build in Docker?

Answer:

A multi-stage build uses multiple FROM instructions in a single Dockerfile. The first stage is a build stage where you install all build tools, compile your code, and build your application. The final stage is a minimal runtime stage where you only copy the compiled output from the first stage. This means your final image does not contain build tools, compilers, or development dependencies — only what is needed to run the app. This significantly reduces the final image size, sometimes from gigabytes down to megabytes.

Q15.

What is the Docker container lifecycle?

Answer:

A Docker container goes through several states during its lifetime. Created — the container exists but has not started yet. Running — the container is actively executing. Paused — the container is temporarily suspended but not stopped. Stopped — the container was running but has been stopped gracefully. Exited — the container has finished its task or crashed. You can move between these states using commands like `docker start`, `docker stop`, `docker pause`, `docker unpause`, and `docker rm` to remove a container completely.

SECTION 3 — Scenario Based Questions

Q16.

What is Docker Compose and when would you use it?

Answer:

Docker Compose is a tool for defining and running multi-container applications using a single YAML file called `docker-compose.yml`. Instead of running multiple `docker run` commands with many flags, you define all your services, networks, and volumes in one file and start everything with a single command: `docker-compose up`. It is perfect for local development when you have multiple services like a web app, database, and cache that need to work together. For production at scale, you would use Kubernetes instead.

Q17.

A Docker container keeps crashing and restarting. How do you troubleshoot it?

Answer:

First, check the container logs using `docker logs` followed by the container name or ID. Add the `-f` flag to follow logs in real time. Look for error messages that explain why it is crashing. Next, use `docker inspect` on the container to see detailed information about its configuration, environment variables, and network settings. Use `docker stats` to check if the container is running out of memory or CPU. If a dependency is missing, update your `Dockerfile` to include it, rebuild the image, and redeploy.

Q18.

How would you set up a multi-container application using Docker Compose?

Answer:

In your `docker-compose.yml` file, you define each service separately. For a web application with a database and cache, you would have three services: one for the web app, one for the database like PostgreSQL, and one for the cache like Redis. Each service specifies its image, ports, environment variables, and volumes. Docker Compose automatically creates a shared network so all services can communicate using their service names as hostnames. You add a named volume for the database to ensure data persists. Then run `docker-compose up` to start everything together.

Q19.

Your Docker image is 2GB. How do you reduce its size?

Answer:

There are several ways to reduce image size. First, use a multi-stage build to separate the build stage from the runtime stage, so build tools do not end up in the final image. Second, use a lightweight base image like Alpine instead of Ubuntu or Debian — Alpine is only about 5MB. Third, combine all `RUN` commands into a single instruction to minimize layers. Fourth, add a `.dockerignore` file to exclude unnecessary files like `node_modules`, `.git`, and test files from being copied into the image. These steps together can reduce image size dramatically.

Q20.

What is port mapping and what is the difference between exposing and publishing a port?

Answer:

Port mapping connects a port inside the container to a port on the host machine so external traffic can reach the container. `EXPOSE` in a `Dockerfile` is just a declaration — it tells Docker which port the application listens on inside the container, but it does not actually open that port to the outside world. Publishing a port, using the `-p` flag with `docker run`, actually maps the container port to a host port so external traffic can reach it. For example, `-p 8080:80` maps port 80 inside the container to port 8080 on your host machine.

Q21.

What are Docker image tags and why are they important?

Answer:

A Docker image tag is a label that identifies a specific version of an image. For example, `myapp:1.0`, `myapp:2.0`, or `myapp:latest`. Tags are important because they let you manage multiple versions of your application. When you deploy, you know exactly which version you are running. If something goes wrong with version 2.0, you can easily rollback to version 1.0. Never rely on the latest tag in production because it changes every time you push a new image, which can cause unexpected behavior.

Q22.

What are environment variables in Docker and how do you pass them?

Answer:

Environment variables are key-value pairs that configure your application at runtime without hardcoding values in the code. There are multiple ways to pass them to a container. Using the `-e` flag with `docker run`, like `-e DB_PASSWORD=secret`. Using an env file with `--env-file myfile.env`. Using the `ENV` instruction in the Dockerfile to set default values. In Docker Compose, you can use the `environment` key to list variables or the `env_file` key to load from a file. Never hardcode sensitive values like passwords — always use environment variables or secrets management.

SECTION 4 — Hard / Advanced Questions

Q23.

Explain Docker image layers and layer caching.

Answer:

Every instruction in a Dockerfile creates a read-only layer. These layers are stacked on top of each other to form the final image. When you build an image, Docker checks if a layer has changed since the last build. If nothing has changed, Docker reuses the cached version of that layer instead of rebuilding it. This makes subsequent builds much faster. If you change an instruction, Docker rebuilds that layer and all layers after it. This is why you should put instructions that change frequently, like copying your application code, at the end of the Dockerfile, and stable instructions like installing packages near the top.

Q24.

What is Docker content trust?

Answer:

Docker content trust is a security feature that uses digital signatures to verify that Docker images are authentic and have not been tampered with. When you push an image, it gets signed with a private key. When someone pulls that image, Docker verifies the signature to make sure it came from a trusted source and was not modified by anyone in between. This protects against malicious images being pushed with the same name as a legitimate image. You enable it by setting the environment variable `DOCKER_CONTENT_TRUST=1`.

Q25.

What is container immutability and why is it important?

Answer:

Immutability means that once a container is built and running, you do not make changes directly inside it. If you need to update the application, you make changes to the Dockerfile, rebuild the image, and deploy a new container. You never SSH into a running container and modify files. This is important because it ensures consistency — everyone running the same image gets the exact same environment. There are no surprise differences between development, staging, and production. If something goes wrong, you know exactly what changed because all changes go through the Dockerfile.

Q26.

What is the Docker daemon and how does the client-server architecture work?

Answer:

The Docker daemon is a background service that runs on your machine and does all the actual work — managing containers, images, networks, and volumes. The Docker CLI is the client that you use to type commands. When you run a docker command like `docker run` or `docker build`, the CLI sends that request to the Docker daemon through a REST API. The daemon processes the request and performs the action. This is a client-server architecture where the CLI is the client and the daemon is the server. They communicate using a Unix socket or TCP connection.

Q27.

What is Docker and Kubernetes? When do you use one over the other?

Answer:

Docker is a tool for creating and running containers. It packages your application into a container image and runs it. Kubernetes is a container orchestration platform that manages many containers across many servers. For small applications running on a single server or a few servers, Docker alone with Docker Compose is enough. But when you need to deploy at large scale across many servers, automatically scale containers up and down based on traffic, recover from failures automatically, and manage complex deployments with zero downtime — that is when you need Kubernetes. Docker creates the containers, Kubernetes manages them at scale.

Q28.

What are common Docker networking issues and how do you troubleshoot them?

Answer:

Common issues include containers not being able to communicate with each other, containers losing their IP address when restarted, and containers not being able to reach the internet. To troubleshoot, use `docker inspect` on the container to see its network configuration and IP address. Use `docker network ls` to list all networks and `docker network inspect` to see which containers are connected to a network. If two containers cannot communicate, check that they are on the same network. If a container cannot reach the internet, check firewall rules and DNS settings on the host machine.

Q29.

How do you design a complete Docker solution for a microservices application?

Answer:

Start by creating individual Dockerfiles for each microservice — web server, API, database, cache, and monitoring. Write a `docker-compose.yml` file for local development that defines all services with their ports, environment variables, volumes, and networking. Each service communicates with others using the service name as the hostname. Use named volumes for databases to persist data. For production, deploy to Kubernetes where each microservice becomes a deployment with its own pods, services, and scaling rules. Add Prometheus for metrics collection and Grafana for dashboards to monitor the health of all services.

Q30.

What is a Docker health check and how do you configure it?

Answer:

A Docker health check is a command that Docker runs periodically inside the container to verify that the application is working correctly. If the health check fails repeatedly, Docker marks the container as unhealthy and can automatically restart it. You configure it in the Dockerfile using the `HEALTHCHECK` instruction. You specify the command to run, how often to run it, how long to wait for a response, and how many failures before marking it unhealthy. For a web application, the health check might be a simple HTTP request to the application endpoint to verify it is responding.